PROJET 1 BASE DE DONNÉES

PLATEFORME DE STREAMING MUSICAL

THE WAVE

Formation :

L2 Informatique

Binôme :

Manon QUIVY

Laeticia KHRIS

Année universitaire :

2025–2026

1. Présentation générale du projet

Le projet **The Wave** consiste à concevoir et réaliser une plateforme de distribution de musique à la demande. Cette application web permet à des utilisateurs inscrits d'écouter des morceaux en ligne, de consulter des informations détaillées sur les artistes, groupes, albums et morceaux, de créer et partager des playlists, ainsi que de recevoir des suggestions personnalisées basées sur leur historique d'écoute et leurs interactions sociales.

L'objectif principal du projet est de mettre en œuvre l'ensemble des étapes de conception d'une base de données relationnelle et de son exploitation via une application web dynamique développée avec Flask et PostgreSQL.

1.1 Fonctionnalités principales

Le site web The Wave propose notamment :

- La création de comptes utilisateurs et l'authentification sécurisée ;
- La consultation d'un catalogue musical riche (groupes, artistes, albums, morceaux) ;
- L'écoute de morceaux et d'albums, avec enregistrement des écoutes ;
- La création, l'édition et la suppression de playlists (publiques ou privées) ;
- Une page d'accueil mettant en avant les contenus populaires ;
- Un moteur de recherche (morceaux, albums, playlists) ;
- Une page de profil personnalisée ;
- Un système de suggestions musicales.

1.2 Manuel utilisateur

- **Connexion / Inscription** :
    - L'utilisateur crée un compte avec un pseudonyme unique, une adresse e-mail et un mot de passe.
    - Il peut se connecter ensuite avec ces identifiants.
- **Accueil** :
    - Affichage des albums récents, des groupes les plus suivis et des morceaux les plus écoutés de la semaine.
    - Cliquer sur les titres d'albums ou de morceaux et sur les noms de groupes pour accéder à leur page correspondante.
- **Navigation** :
    - Accéder aux différentes pages via la barre de navigation en haut : Accueil, Recherche, Profil, Suggestions.

- **Recherche** :
    - Rechercher des morceaux, albums ou playlists via la barre de recherche en utilisant un mot-clé, un titre ou un nom de groupe.
    - Les résultats s'affichent tous et peuvent être filtrés par type (Morceau, Album, Playlist).
- **Écoute** :
    - Écouter un morceau ou un album depuis sa page dédiée en cliquant sur le lien pour écouter.
    - Chaque écoute est enregistrée afin d'alimenter l'historique et les suggestions.
- **Profil** :
    - Consulter les statistiques personnelles, l'historique d'écoute et les actualités.
- **Playlists** :

  Créer, modifier ou supprimer des playlists personnelles depuis la page Profil.
- **Suggestions** :
    - Découvrir de nouveaux contenus adaptés aux goûts de l'utilisateur, basés sur l'historique d'écoute et les playlists publiques.

2. Conception de la base de données

2.1 Analyse du cahier des charges

Le cahier des charges impose la gestion de plusieurs entités fortement liées : artistes, groupes, albums, morceaux et utilisateurs. Il met également en avant des relations complexes, telles que : l'appartenance multiple et temporelle des artistes à des groupes ; la participation d'artistes à des morceaux indépendamment de leur groupe ; la notion de groupes « solo » pour les artistes ayant une carrière individuelle ; des relations sociales entre utilisateurs et avec les groupes.

2.2 Modèle conceptuel (entité-association)

Le modèle conceptuel repose sur les entités principales suivantes :

- **Artiste** : informations biographiques ;
- **Groupe** : identité musicale et caractéristiques générales ;
- **Album** : collection ordonnée de morceaux ;
- **Piste (morceau)** : unité musicale écoutable ;

- **Utilisateur** : compte permettant l'accès à la plateforme ;
- **Playlist** : liste ordonnée de morceaux créée par un utilisateur.

Plusieurs associations permettent de répondre aux contraintes du sujet :

- **appartient** : association ternaire entre artiste, groupe et rôle, avec dates d'arrivée et de départ ;
- **participe** : association entre artiste et piste pour gérer les collaborations et invités ;
- **interprète** : association entre groupe et piste ;
- **publie** : association entre groupe et album ;
- **comprend** : association ordonnée entre playlist et piste ;
- **écoute** : historique des écoutes des utilisateurs ;
- **suit_groupe** et **suit_utilisateur** : fonctionnalités sociales.

2.3 Hypothèses et choix de modélisation

- Les artistes ayant une carrière solo sont modélisés comme des groupes distincts dont ils sont l'unique membre.
- Les rôles (chanteur, guitariste, etc.) sont factorisés dans une table dédiée afin d'éviter les redondances.
- L'ordre des morceaux dans un album ou une playlist est géré par un attribut numérique.
- Certaines contraintes métiers (par exemple l'impossibilité pour un artiste de participer à deux groupes simultanément avec le même rôle) ne sont pas entièrement garanties par la base et sont supposées respectées par l'application.

3. Modèle relationnel et implémentation

3.1 Traduction relationnelle

Le modèle conceptuel a été traduit en un schéma relationnel implémenté sous PostgreSQL. Chaque entité est représentée par une table, et les associations par des tables de jointure munies de clés primaires composites.

Exemples de tables principales :

- **artiste(id_artiste, nom, prenom, date_naissance, date_mort, image)** ;
- **groupe(id_groupe, nom, date_creation, nationalite, genre, image)** ;
- **album(id_album, titre, date_parution, im_couverture, description)** ;
- **piste_lecture(id_piste, titre, duree, paroles, id_album, num_piste)** ;
- **utilisateur(pseudonyme, email, date_inscription, mot_de_passe)**.

3.2 Contraintes et intégrité

- Les clés primaires assurent l'unicité des enregistrements.
- Les clés étrangères garantissent la cohérence entre les tables (ON DELETE CASCADE ou SET NULL selon les cas).
- Des contraintes CHECK sont utilisées pour vérifier la cohérence temporelle (dates d'arrivée et de départ).
- L'unicité de la position des morceaux dans une playlist ou un album est assurée.

Certaines règles plus complexes (par exemple la gestion avancée des conflits temporels) sont traitées au niveau applicatif.

4. Site web et exploitation de la base

4.1 Architecture technique

Le site est développé en Python avec le micro-framework Flask. La communication avec la base de données PostgreSQL est assurée par psycopg2. Les pages HTML sont générées à l'aide de templates Jinja2.

L'architecture des fichiers respecte les consignes :

- main.py : routes Flask et logique applicative ;
- db.py : connexion à la base de données ;
- templates/ : templates HTML ;
- static/ : fichiers statiques ;
- dump.sql : dump complet de la base ;
- rapport.pdf : documentation du projet.

4.2 Sécurité et authentification

- Les mots de passe des utilisateurs sont **hachés** avec l'algorithme **bcrypt**, ce qui assure qu'ils ne sont jamais stockés en clair dans la base de données.

- Les **sessions Flask** permettent de maintenir l'authentification entre les pages, de manière sécurisée et persistante tant que l'utilisateur est connecté.

4.3 Suggestions personnalisées

Le système de recommandations repose sur plusieurs critères complémentaires :

- Historique d'écoute : analyse les morceaux et artistes les plus fréquemment écoutés par l'utilisateur.
- Artistes favoris : propose d'autres morceaux des artistes les plus écoutés.
- Comportement similaire d'autres utilisateurs : détecte les groupes ou playlists appréciés par des utilisateurs aux goûts proches.
- Playlists publiques : recommande des playlists contenant des morceaux déjà écoutés par l'utilisateur.

But : proposer un contenu varié et pertinent, cohérent avec les goûts et les habitudes d'écoute de l'utilisateur.

5. Exemples de requêtes SQL significatives

- **Accueil - morceaux les plus écoutés sur 7 jours** :

```
SELECT titre, count(id_ecoute) AS nb
FROM piste_lecture NATURAL JOIN ecoute
WHERE date_ecoute >= CURRENT_DATE – 7
GROUP BY titre ORDER BY nb DESC LIMIT 5;
```

Fonctionnement :

- Associe chaque piste avec ses écoutes enregistrées dans la table ecoute grâce au NATURAL JOIN.
- Conserve les écoutes des 7 derniers jours (WHERE)
- Regroupe les enregistrements par titre de piste (GROUP BY) et compte le nombre total d'écoutes par piste (COUNT).
- Trie les morceaux du plus écouté au moins écouté et conserve uniquement les 5 premiers. (ORDER BY et LIMIT)

Utilité : afficher sur la page d'accueil les morceaux les plus populaires de la semaine.

Playlists publiques contenant des morceaux déjà écoutés :

```
SELECT id_playlist, titre, pseudonyme, COUNT (*) AS nb
FROM playlist NATURAL JOIN comprend
WHERE visibilite = TRUE AND id_piste IN (
    SELECT DISTINCT id_piste FROM ecoute WHERE pseudonyme = %s
)
GROUP BY id_playlist, titre, pseudonyme
ORDER BY nb DESC
LIMIT 5;
```

Fonctionnement :

- La **sous-requête** sélectionne toutes les pistes distinctes déjà écoutées par l'utilisateur.
- Le WHERE permet de ne considérer que les playlists publiques contenant **au moins un morceau déjà écouté** par l'utilisateur.
- Pour chaque playlist on compte le nombre de pistes connues par l'utilisateur (COUNT et GROUP BY)
- Trie et limite pour afficher les 5 playlists ayant le plus de pistes déjà écoutées.

Utilité : proposer à l'utilisateur des playlists pertinentes basées sur son historique d'écoute.

Vues SQL : des vues ont été créées pour pré-calculer la popularité des morceaux et des indicateurs agrégés, simplifiant les requêtes côté application et optimisant les performances.

6. Améliorations possibles

Plusieurs pistes d'amélioration peuvent être envisagées :

- intégration d'un véritable lecteur audio avec fichiers multimédias ;
- ajout de commentaires et de likes sur les morceaux et playlists ;

- amélioration des algorithmes de recommandation (pondération, apprentissage)
- optimisation des performances via des index supplémentaires ;
- gestion avancée des droits et des rôles utilisateurs.

7. Organisation du travail

Le travail a été réalisé en binôme de manière équilibrée :

- **Manon QUIVY :**

    - Développement des pages Accueil, Morceaux, Albums, Groupes et Profil.
    - Intégration des images et mise en forme du site avec CSS.

- **Laeticia KHRIS :**

    - Développement des pages Suggestions et Recherche.
    - Gestion des fonctionnalités de connexion, inscription et des sessions utilisateur.
    - Gestion du hachage et de la sécurité des mots de passe.

Conclusion

Le projet *The Wave* a été l'occasion de mettre en application les notions étudiées en bases de données et en programmation web. La conception du schéma, son implémentation en PostgreSQL ainsi que son exploitation via une application web ont permis de mieux comprendre le lien entre la modélisation des données et leur utilisation concrète dans un site fonctionnel.

Ce projet nous a également permis de travailler sur des aspects essentiels tels que la gestion des relations complexes, l'intégrité des données et l'organisation d'un

projet en binôme. Enfin, *The Wave* constitue une base solide qui pourrait être enrichie par la suite, notamment par l’ajout de nouvelles fonctionnalités ou par l’amélioration des performances et de l’ergonomie du site.